



```
In [26... import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
In [27... df=pd.read_csv("car_acceptability.csv")
df
```

```
Out[27]:
```

	buying price	maintenance cost	number of doors	number of persons	lug_boot	safety	evaluation
0	vhigh	vhigh	2	2	small	med	unacc
1	vhigh	vhigh	2	2	small	high	unacc
2	vhigh	vhigh	2	2	med	low	unacc
3	vhigh	vhigh	2	2	med	med	unacc
4	vhigh	vhigh	2	2	med	high	unacc
...	...	...	...	...	...	...	...
1722	low	low	5	5	med	med	good
1723	low	low	5	5	med	high	vgood
1724	low	low	5	5	big	low	unacc
1725	low	low	5	5	big	med	good
1726	low	low	5	5	big	high	vgood

1727 rows × 7 columns

```
In [28... le = LabelEncoder()

df["buying price"]=le.fit_transform(df["buying price"])
df["maintenance cost"]=le.fit_transform(df["maintenance cost"])
df["lug_boot"]=le.fit_transform(df["lug_boot"])
df["safety"]=le.fit_transform(df["safety"])
df["evaluation"]=le.fit_transform(df["evaluation"])
```

```
In [29... df
```

Out[29]:

	buying price	maintenance cost	number of doors	number of persons	lug_boot	safety	evaluation
0	3	3	2	2	2	2	2
1	3	3	2	2	2	0	2
2	3	3	2	2	1	1	2
3	3	3	2	2	1	2	2
4	3	3	2	2	1	0	2
...	...	...	...	...	...	...	...
1722	1	1	5	5	1	2	1
1723	1	1	5	5	1	0	3
1724	1	1	5	5	0	1	2
1725	1	1	5	5	0	2	1
1726	1	1	5	5	0	0	3

1727 rows × 7 columns

```
In [30... X= df.drop("evaluation",axis=1)
Y=df["evaluation"]
```

```
In [31... X_train,X_test,Y_train,Y_test=train_test_split(X,Y,test_size=0.2,random_s
```

```
In [32... svm_model=SVC(kernel="linear")
svm_model.fit(X_train,Y_train)
```

Out[32]:

SVC		
Parameters		
	C	1.0
	kernel	'linear'
	degree	3
	gamma	'scale'
	coef0	0.0
	shrinking	True
	probability	False
	tol	0.001
	cache_size	200
	class_weight	None
	verbose	False
	max_iter	-1
	decision_function_shape	'ovr'
	break_ties	False
	random_state	None

```
In [33...] Y_pred=svm_model.predict(X_test)
```

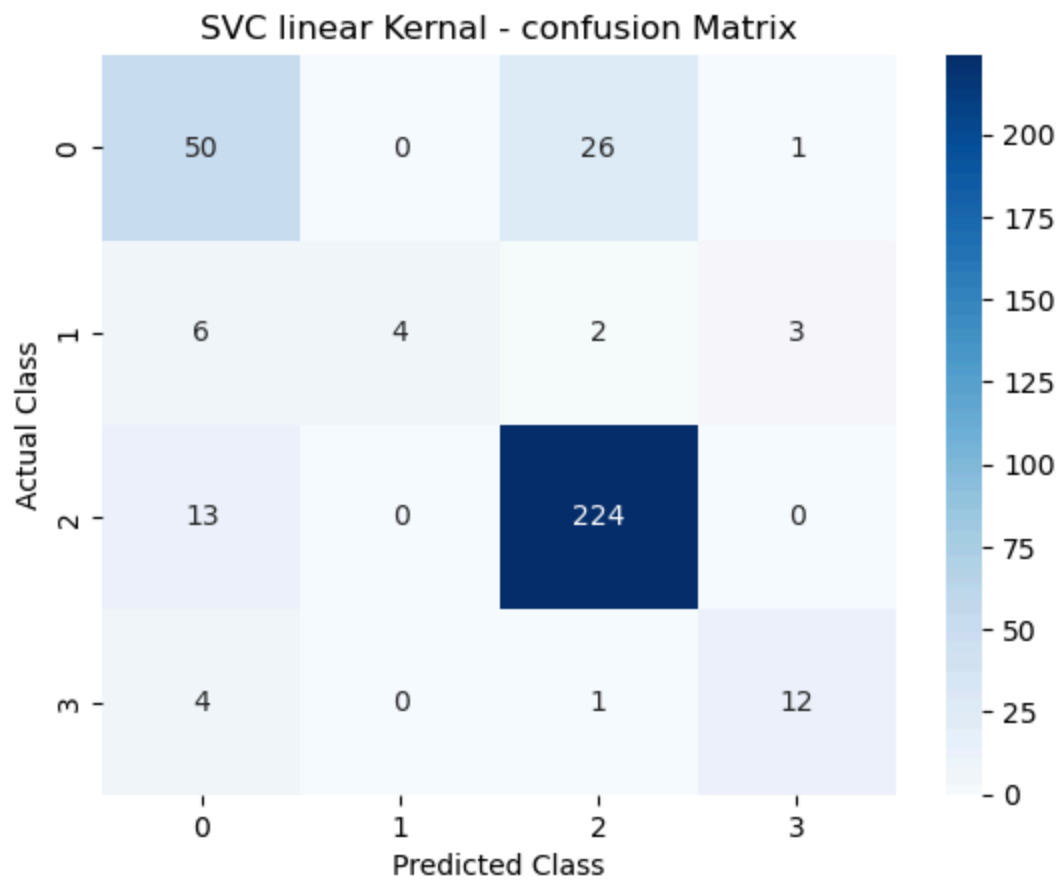
```
In [34...] print("Accuracy:",accuracy_score(Y_test,Y_pred))
cm=confusion_matrix(Y_test,Y_pred)
print("\nConfusion Matrix:\n",confusion_matrix(Y_test,Y_pred))
```

Accuracy: 0.7138728323699421

Confusion Matrix:

```
[[ 12   0  65   0]
 [  0   0  15   0]
 [  2   0 235   0]
 [ 12   0   5   0]]
```

```
In [40...] sns.heatmap(cm,annot=True,fmt="d",cmap="Blues")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("SVC linear Kernal - confusion Matrix")
plt.show()
```



```
In [36... svm_model=SVC(kernel="poly")  
svm_model.fit(X_train,Y_train)
```

Out[36]:

SVC		
Parameters		
	C	1.0
	kernel	'poly'
	degree	3
	gamma	'scale'
	coef0	0.0
	shrinking	True
	probability	False
	tol	0.001
	cache_size	200
	class_weight	None
	verbose	False
	max_iter	-1
	decision_function_shape	'ovr'
	break_ties	False
	random_state	None

```
In [37...] Y_pred=svm_model.predict(X_test)
```

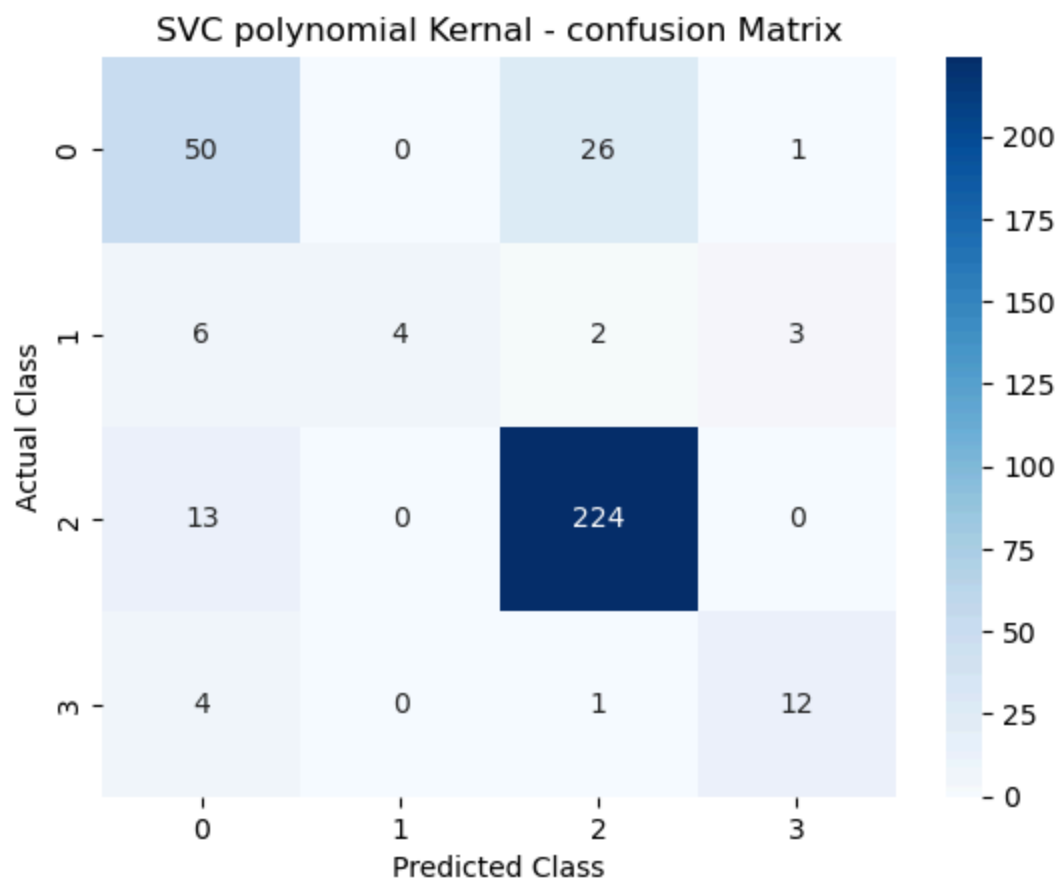
```
In [38...] print("Accuracy:",accuracy_score(Y_test,Y_pred))
cm=confusion_matrix(Y_test,Y_pred)
print("\nConfusion Matrix:\n",confusion_matrix(Y_test,Y_pred))
```

Accuracy: 0.838150289017341

Confusion Matrix:

```
[[ 50   0  26   1]
 [  6   4   2   3]
 [ 13   0 224   0]
 [  4   0   1  12]]
```

```
In [39...] sns.heatmap(cm,annot=True,fmt="d",cmap="Blues")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("SVC polynomial Kernal - confusion Matrix")
plt.show()
```



In []: